

Database design

The choice and why

PrintXpress uses Room on top of SQLite for all storage. Room is the standard local database for native Android, and SQLite is the engine built into every Android phone. This means the whole database lives on the device, the app works offline, and there is no server to set up or pay for.

This is the simplest reliable option for the project, and it is portable. The data is a single SQLite file, so a copy can travel with the app and be opened in a free tool like DB Browser for SQLite for a marker to inspect.

Room must be added as a dependency before any of this code is written. The proposed versions are in [09-technical-documentation.md](#) under setup, and they should be approved first.

How the data is stored on the device

SQLite has a small set of storage classes, so the Kotlin types map onto it like this.

Kotlin type	SQLite type	Used for
<code>Long</code>	INTEGER	Ids and timestamps (epoch milliseconds)
<code>String</code>	TEXT	Names, descriptions, status, file paths

Double	REAL	Prices and totals
Boolean	INTEGER (0 or 1)	Flags such as default address or read state

Every table uses a Long primary key that Room auto-generates. Foreign keys link the tables and use onDelete = CASCADE where a child has no meaning without its parent, for example an order item without its order.

Tables

users

Holds one row per account. Email or phone is the login id, and only the password hash is stored.

Column	Type	Key	Notes
userId	Long	PK	Auto-generated
fullName	String		Required
email	String	unique	Nullable if phone is used
phone	String	unique	Nullable if email is used
passwordHash	String		Hashed, never plain text

createdAt	Long		Epoch millis
-----------	------	--	--------------

addresses

Delivery addresses for a user. One can be the default.

Column	Type	Key	Notes
addressId	Long	PK	Auto-generated
userId	Long	FK to usersdelete	Cascade on delete
label	String		For example home or office
line1	String		Street address
city	String		
postalCode	String		
isDefault	Boolean		Only one true per user

categories

Top level product groups, for example business cards or t-shirts. Seeded on first run.

Column	Type	Key	Notes
categoryId	Long	PK	Auto-generated
name	String		Required
description	String		Short summary
iconRef	String		Name of a drawable or asset

products

A printable product inside a category, with a base price and the option lists.

Column	Type	Key	Notes
productId	Long	PK	Auto-generated
categoryId	Long	FK to categories	Cascade on delete

name	String			Required
description	String			
basePrice	Double			Price for the base specification
material	String			Default material
imageRef	String			Sample preview image
sizeOptions	String			Comma separated for example A6,A5,A-
materialOptions	String			Comma separated for example matte,gloss

Storing the option lists as comma separated text keeps the schema small and is simple for a beginner. The app splits the text into a list when it shows the choices.

orders

One row per placed order. The total is stored so a past order keeps its price even if the catalog changes.

Column	Type	Key	Notes
orderId	Long	PK	Auto-generated
userId	Long	FK to users	Cascade on delete
orderDate	Long		Epoch millis
status	String		processing, printing, ready for pickup, out for delivery, completed, cancelled
fulfilment	String		pickup or delivery
deliveryAddressId	Long	FK to addresses	Nullable, set null on delete, used only for delivery
promoid	Long	FK to promotions	Nullable, set null on delete
totalAmount	Double		Sum of the item line totals after any discount
notes	String		Optional customer note

order_items

The lines inside an order. Each line is one product with its chosen specification and the design or text.

Column	Type	Key	Notes
orderId	Long	PK	Auto-generated
productId	Long	FK to orders	Cascade on delete
quantity	Int		At least one
paperType	String		Chosen material or paper
size	String		Chosen size
unitPrice	Double		Price for one unit at this spec
designPath	String		File path of the upload, nullable
customText	String		Typed text, nullable

saved_designs

Designs a user saved to reuse on later orders.

Column	Type	Key	Notes
designId	Long	PK	Auto-generated
userId	Long	FK to users	Cascade on delete
title	String		Required
filePath	String		Stored file reference
uploadedAt	Long		Epoch millis

notifications

In-app messages for a user, such as order confirmations and offers.

Column	Type	Key	Notes
notificationId	Long	PK	Auto-generated
userId	Long	FK to users	Cascade on delete

title	String			
message	String			
type	String			order or promo
isRead	Boolean			Defaults to false
createdAt	Long			Epoch millis

promotions

Seasonal offers and discount codes. The app shows only active offers based on the dates.

Column	Type		Ke	Notes
promold	Long		PK	Auto-gen
title	String			
description	String			

discountPercent	Int			For example 10 for 10 percent
code	String			Discount code
validFrom	Long			Epoch millis
validTo	Long			Epoch millis

Relationships

The links between tables, read as one parent to many children.

Parent	Child	Type	Meaning
users	addresses	1 to many	A user saves many addresses
users	orders	1 to many	A user places many orders
users	saved_designs	1 to many	A user saves many designs
users	notifications	1 to many	A user receives many notifications

categories	products	1 to many	A category holds many products
orders	order_items	1 to many	An order has many items
products	order_items	1 to many	A product appears on many order lines
addresses	orders	1 to many (optional)	An address is used by delivery orders
promotions	orders	1 to many (optional)	A promotion is applied to many orders

The full picture is in the ER diagram at [diagrams/04-er.drawio](#), explained in [05-diagrams.md](#).

Example entity and DAO

A short example of how a table becomes Room code. This is the pattern the rest of the entities follow.

```
// One row in the users table.
@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true) val userId: Long = 0,
    val fullName: String,
    val email: String?, // email or phone is used as the login id
    val phone: String?,
    val passwordHash: String, // hashed, never plain text
    val createdAt: Long
)

// Reads and writes for the users table.
@Dao
interface UserDao {
    // Save a new user and return the new id.
    @Insert
    suspend fun insert(user: User): Long
```

```
// Find a user by email for login.
@Query("SELECT * FROM users WHERE email = :email LIMIT 1")
suspend fun findByEmail(email: String): User?

// Find a user by phone for login.
@Query("SELECT * FROM users WHERE phone = :phone LIMIT 1")
suspend fun findByPhone(phone: String): User?
}
```

Seeding and the portable file

The starting catalog (categories, products, and sample promotions) is loaded the first time the database is created, using a `RoomDatabase.Callback`. This is the most reliable choice for a beginner because there is no pre-built file to keep in sync with the schema.

To hand a filled database to a marker, export the SQLite file after the app has run once. Steps are in [09-technical-documentation.md](#). If a pre-filled file is preferred instead of the callback, place it in `app/src/main/assets/database/printxpress.db` and load it with Room's `createFromAsset`. Use one approach only, do not mix the two.